Reviewer Pack — Minimal Affine Plane Curve Degree Correlation with Communica…

Entry 9b94797d599d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 05:01:18 UTC

Minimal Affine Plane Curve Degree Correlation with Communication Complexity

Entry ID: `9b94797d599d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 05:01:18 UTC

1. Conjecture

Field A (mathematical branch): Affine Geometry **Field B** (complexity object): Communication Complexity

Statement:

> The minimal degree of an affine plane curve corresponding to a given Boolean function is linearly correlated with the communication complexity of that function, such that the minimal degree D(C) = Θ(Comm(f)) for all boolean functions f.

Rationale (proposer's reasoning):

> Affine geometry provides a geometric representation of boolean functions, and its invariants could potentially reveal underlying structural properties. The correlation with communication complexity might expose new insights into the interplay between algebraic structures and the complexity of information exchange.

Taxonomy category: `AffineGeometryCommunicationComplexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `9523f979409ed4a8`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a given Boolean function, if the correlation coefficient between the minimal curve degree D(C) and communication complexity Comm(f) is greater than or equal to 0.8 across at least 30 seeds, then the conjecture is supported.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Minimal Affine Plane Curve Degree and Communication Complexity` - `Affine Geometry in relation to Communication Complexity of Boolean Functions` - `Degree Correlation between Affine Plane Curves and Communication Complexity`

Top relevant hits considered: - [http://arxiv.org/abs/1706.01577v5] Characterization of Spherical and Plane Curves Using Rotation Minimizing Frames - [http://arxiv.org/abs/1609.01155v1] The Transform of a line of Desargues Affine Plane in an additive Group of its Points - [http://arxiv.org/abs/1808.03244 Higher Order Degrees of Affine Plane Curve Complements - [http://arxiv.org/abs/2406.18700v4] Structure of sparse Boolean functions over Abelian groups, and its application to testing - [http://arxiv.org/abs/0903.3848v1] Join-irreducible Boolean functions - [http://arxiv.org/abs/0802.4323v1] Non-singular affine surfaces with self-maps - [http://arxiv.org/abs/math/0503162v2] Plane curves and contact geometry - [http://arxiv.org/abs/2106.15501v3] Ramblings on the freeness of affine hypersurfaces

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=241.1s

5.1 Generated Python source

```python
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity(f):
        n = len(f)
        if n == 1:
            return 1
        if n == 2:
            return 2
        if n == 3:
            return 4
        # For larger n, use a simple DPLL solver (simplified version)
        def dpll(clauses, assignment):
            if not clauses:
                return True
            unit_clause = next((c for c in clauses if len(c) == 1), None)
            if unit_clause:
                literal = unit_clause[0]
```

```
            new_assignment = assignment.copy()
            new_assignment[literal] = True
            if dpll([c for c in clauses if literal not in c], new_assignment):
                return True
            new_assignment[literal] = False
            if dpll([c for c in clauses if -literal not in c], new_assignment):
                return True
            return False
        pure_literal = next((l for l in range(1, n+1) if (l not in assignment and -l not in as
        if pure_literal:
            new_assignment[pure_literal] = True
            if dpll(clauses, new_assignment):
                return True
            new_assignment[pure_literal] = False
            if dpll(clauses, new_assignment):
                return True
            return False
        literal = random.choice([l for l in range(1, n+1) if l not in assignment and -l not in
        new_assignment[literal] = True
        if dpll(clauses, new_assignment):
            return True
        new_assignment[literal] = False
        if dpll(clauses, new_assignment):
            return True
        return False

    def to_clauses(f):
        n = len(f)
        clauses = []
        for i in range(2**n):
            clause = []
            for j in range(n):
                if (i >> j) & 1:
                    clause.append(j + 1)
                else:
                    clause.append(-(j + 1))
            clauses.append(clause)
        return clauses

    clauses = to_clauses(f)
    assignment = {}
    return len(assignment) if dpll(clauses, assignment) else n

def minimal_affine_curve_degree(f):
    n = len(f)
    # Simplified procedure to estimate the degree (not actual affine curve construction)
    return n // 2

results = []
for _ in range(30):
    n = random.choice([5, 10, 15, 20, 30, 40])
    f = generate_boolean_function(n)
    comm_complexity = communication_complexity(f)
    degree = minimal_affine_curve_degree(f)
    results.append((comm_complexity, degree))
```

```
    if not results:
        return {
            "metric_name": "communication_rank",
            "metric_value": 0,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "empty_results"
        }

    comm_ranks = [comm for comm, _ in results]
    degrees = [deg for _, deg in results]
    correlation = sum((comm - sum(comm_ranks) / len(comm_ranks)) * (deg - sum(degrees) / len(degre
    correlation /= (len(results) * sum((comm - sum(comm_ranks) / len(comm_ranks))**2 for comm in c

    return {
        "metric_name": "communication_rank",
        "metric_value": correlation,
        "instances_tested": 30,
        "n_max": max(n for _, n in results),
        "conjecture_holds": correlation >= 0.8,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 31)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if not results:
        print("RESULT: INCONCLUSIVE reason=empty_results")
    else:
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_value = (sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))**0.
        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_
        elif any(not r["conjecture_holds"] for r in results):
            first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds
            print(f"RESULT: FALSIFIED counterexample=\"correlation_threshold_not_met\" first_faili
        else:
            print("RESULT: INCONCLUSIVE reason=insufficient_support")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, which means the pre-registered support condition could not be unambiguously met. | next: Retry the experiment with a longer timeout or increase the number of seeds to ensure sufficient data is collected for analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16784
2	preregistration	ollama_remote	glm4:latest	0	0	9015
3	novelty	ollama_remote	glm4:latest	0	0	8152
4	novelty	ollama_remote	glm4:latest	0	0	10417
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17385
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9439
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11300
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	35836
9	judge	ollama_remote	glm4:latest	0	0	61369

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 179699 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/9b94797d599d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9b94797d599d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/9b94797d599d.tar.gz` (if generated)